

CS FUNDAMENTALS

GENERATED BY KALYX CURRICULUM INTELLIGENCE PIPELINE

COURSE OVERVIEW & SYLLABUS DESCRIPTION

Computer Science Fundamentals introduces the core concepts that form the foundation of modern computing systems. The course covers computer organization, operating systems, programming fundamentals, data structures, algorithms, databases, computer networks, software engineering principles, and cybersecurity basics. Students will learn problem-solving techniques, computational thinking, and the design of efficient software systems. Through practical examples and hands-on exercises, learners will develop a strong

METADATA PROFILE

Package Type: Full Classroom Implementation Package

Generated Slide Count: 20 fully-illustrated slides

Assessment Bank size: 20 cognitive evaluation items

Pedagogical Standard: Bloom's Revised Taxonomy (Tiers 1-6)

SECTION 1: INSTRUCTIONAL SLIDE DECK & NOTES

SLIDE 1: CORE COMPUTER SCIENCE FOUNDATIONS: COURSE OVERVIEW

- Comprehensive review of core CS pillars: DBMS, SQL, Computer Networks, OOPS, and OS
- DBMS: Relational schemas, ACID transactions, Normalization, indexing, and recovery
- Computer Networks: Physical communication, OSI/TCP layers, routing, flow control, and APIs
- OOP & OS: Object methodologies, SOLID principles, kernel states, memory virtualization, and deadlocks
- Course Goal: Cultivate industry-grade design capabilities and technical depth in software infrastructure

VISUAL COMPANION COMPONENT SUGGESTION:

Visual Layout (Sleek Dark Mode): Abstract grid network connecting Database, Server, and Client nodes.

LECTURE SPEAKER NOTES & PEDAGOGY:

Talking Points:

- Academic Formulation: Introduce the five core pillars of Computer Science foundations: DBMS, SQL, Computer Networks, OOPS, and OS.
- Academic Formulation: Explain that these pillars form the engineering bedrock for any high-performance, enterprise-grade software system.
- Academic Formulation: Discuss how databases manage persistent state, while networks handle distributed communications.
- Academic Formulation: Explain the role of Object-Oriented design in managing code complexity and scaling application architectures.
- Academic Formulation: Highlight how Operating Systems manage physical hardware resources and orchestrate CPU scheduling.
- Academic Formulation: Outline the lesson roadmap, setting expectations for academic rigor, exams, and practical code labs.

Pedagogical Tip:

Do an icebreaker by asking students which layer of the stack they find most interesting (data, network, design, or hardware) and why.

Classroom Examples:

- > "A banking app: uses DBMS for balances, Networks for transactions, OOP for business logic, and OS for scheduling processes."
- > "Web browser: network sockets fetch HTML, OS manages process rendering tabs, OOP organizes UI code."
- > "Smart home devices: micro-OS controls sensors, light network sends packets, databases log historical metrics."
- > "E-commerce server: SQL retrieves inventory, TCP socket sends payload, OOP patterns organize checkout loops."

SLIDE 2: DBMS: THE RELATIONAL MODEL & SCHEMA DESIGN

- Relational Model: Data represented as mathematical relations (tables) with rows (tuples) and columns (attributes)
- Data Integrity: Enforcing domain constraints, entity integrity (primary keys), and referential integrity (foreign keys)
- Functional Dependencies: Determining attributes mapping $X \rightarrow Y$ to identify redundancy
- Schema Normalization: Process of decomposing tables to achieve 1NF, 2NF, 3NF, and BCNF structures
- Anomalies Management: Preventing insertion, update, and deletion discrepancies via schema decomposition

VISUAL COMPANION COMPONENT SUGGESTION:

Visual Layout (Sleek Dark Mode): Entity-Relationship model schema diagram highlighting Primary Key to Foreign Key relationships.

LECTURE SPEAKER NOTES & PEDAGOGY:

Talking Points:

- Academic Formulation: Explain the Relational Model introduced by Edgar F. Codd, representing data as tables or mathematical relations.
- Academic Formulation: Clarify the distinction between physical storage and logical relations in DB design.
- Academic Formulation: Detail the three types of database constraints: entity integrity, referential integrity, and domain constraints.
- Academic Formulation: Explain how functional dependencies (FDs) act as semantic assertions about the attributes in a database.
- Academic Formulation: Walk through the steps of Normalization: from eliminating repeating groups (1NF) to removing partial dependencies (2NF).
- Academic Formulation: Analyze how Third Normal Form (3NF) and Boyce-Codd Normal Form (BCNF) remove transitive functional dependencies.

Pedagogical Tip:

Draw an unnormalized database table on the board and ask students to point out update, insert, and delete anomalies.

Classroom Examples:

- > "An Order table with repeated product rows violating 1NF."
- > "A student course table where student name depends only on student ID, violating 2NF."
- > "A employee department manager table where manager name transitively depends on employee ID through department code, violating 3NF."
- > "A library system table with overlapping candidate keys violating BCNF."

SLIDE 3: DBMS: ACID PROPERTIES & TRANSACTION INTEGRITY

- Atomicity: Ensures all operations within a transaction complete successfully, or all are rolled back
- Consistency: Guarantees that a transaction transforms the database from one valid state to another
- Isolation: Execution of concurrent transactions does not interfere with each other (transaction schedules)
- Durability: Completed transactions are written to persistent storage, surviving subsequent power outages
- State Transitions: Mapping transaction states from active to committed, failed, or aborted

VISUAL COMPANION COMPONENT SUGGESTION:

Visual Layout (Sleek Dark Mode): Block diagram outlining Transaction states: Active, Partially Committed, Committed, Failed, Aborted.

LECTURE SPEAKER NOTES & PEDAGOGY:

Talking Points:

- Academic Formulation: Introduce the concept of a Transaction: a logical unit of database work containing one or more SQL commands.
- Academic Formulation: Detail Atomicity: the 'all or nothing' guarantee managed by the database rollback segments.
- Academic Formulation: Discuss Consistency: how transactions must preserve all active database schemas, check constraints, and triggers.
- Academic Formulation: Explain Isolation: preventing concurrent transactions from viewing intermediate, uncommitted states.
- Academic Formulation: Explain Durability: guaranteeing that once a transaction commits, its updates survive subsequent system crashes.
- Academic Formulation: Trace the life cycle of a transaction: active, partially committed, committed, failed, and aborted states.

Pedagogical Tip:

Draw a database crash scenario mid-transaction and ask students how the recovery manager knows what to undo.

Classroom Examples:

- > "A bank transfer: debiting account A must be atomic with crediting account B."
- > "Consistency check: preventing account balances from dropping below zero due to database constraints."
- > "Isolation isolation: two concurrent checkouts reading the same item stock quantity."
- > "Durability check: changes saved to the write-ahead transaction log before power loss."

SLIDE 4: DBMS: DATABASE CONCURRENCY CONTROL & RECOVERY

- Concurrency Anomalies: Dirty reads, non-repeatable reads, phantom reads, and lost updates
- Lock-Based Protocols: Two-Phase Locking (2PL) ensuring serializability, and Strict 2PL to prevent cascading aborts
- Timestamp Ordering: Assigning unique execution timestamps to resolve write-read conflicts
- Write-Ahead Logging (WAL): Write changes to transaction log on disk before modifying database blocks
- ACID Recovery Manager: Replaying log operations (REDO/UNDO) upon database system startup

VISUAL COMPANION COMPONENT SUGGESTION:

Visual Layout (Sleek Dark Mode): Timeline chart comparing serial execution vs interleaved execution schedules.

LECTURE SPEAKER NOTES & PEDAGOGY:

Talking Points:

- Academic Formulation: Explain the concurrency anomalies that occur when isolation levels are relaxed.
- Academic Formulation: Define a Dirty Read (reading uncommitted data), a Non-Repeatable Read, and a Phantom Read.
- Academic Formulation: Introduce lock-based concurrency control: Shared locks (S) for reads, Exclusive locks (X) for writes.
- Academic Formulation: Explain the Two-Phase Locking (2PL) protocol: growing phase (acquiring locks) and shrinking phase (releasing locks).
- Academic Formulation: Explain why Strict 2PL is required: releasing exclusive locks only at transaction commit prevents cascading aborts.
- Academic Formulation: Introduce Write-Ahead Logging (WAL) and the ACID recovery manager's log-and-replay algorithm.

Pedagogical Tip:

Simulate a deadlock on the whiteboard by locking two student tables in opposing orders.

Classroom Examples:

- > "Dirty Read: reading a balance change that gets rolled back."
- > "Non-Repeatable Read: reading a row, another transaction updates it, reading again yields a different value."
- > "Phantom Read: querying a count of active orders, a new order is inserted, querying again yields a different count."
- > "WAL: a database server crashes, and the system uses the WAL log on startup to REDO committed changes."

SLIDE 5: SQL: DATA DEFINITION VS MANIPULATION LANGUAGE

- DDL (Data Definition Language): Structural statements like CREATE, ALTER, DROP, and TRUNCATE
- DML (Data Manipulation Language): Data-level operations like SELECT, INSERT, UPDATE, and DELETE
- DCL and TCL: Managing access permissions (GRANT/REVOKE) and transaction states (COMMIT/ROLLBACK)
- Constraints mapping: Applying UNIQUE, NOT NULL, DEFAULT, and CHECK constraints to tables
- Savepoint Management: Implementing sub-transaction checkpoints to roll back partial transaction steps

VISUAL COMPANION COMPONENT SUGGESTION:

Visual Layout (Sleek Dark Mode): Dual-pane code layout comparing a table creation statement vs an insertion query.

LECTURE SPEAKER NOTES & PEDAGOGY:**Talking Points:**

- Academic Formulation: Differentiate between Data Definition Language (DDL) and Data Manipulation Language (DML).
- Academic Formulation: Explain that DDL statements modify the database catalog (structure) and cause implicit commits in most engines.
- Academic Formulation: Explain that DML statements modify actual table records and run within active transaction scopes.
- Academic Formulation: Discuss Data Control Language (DCL) commands like GRANT and REVOKE for role-based access security.
- Academic Formulation: Explain Transaction Control Language (TCL) commands: COMMIT, ROLLBACK, and SAVEPOINT.
- Academic Formulation: Highlight how constraints (NOT NULL, UNIQUE, CHECK, FOREIGN KEY) are defined in DDL schemas.

Pedagogical Tip:

Ask students why DDL changes cannot usually be rolled back in standard RDBMS engines.

Classroom Examples:

- > "DDL query: CREATE TABLE students (id INT PRIMARY KEY, email VARCHAR(100) UNIQUE)."
- > "DML query: INSERT INTO students (id, email) VALUES (1, 'test@kalyx.com')."
- > "DCL query: GRANT SELECT ON students TO professor_role."
- > "TCL query: SAVEPOINT checkpoint1; ROLLBACK TO checkpoint1;."

SLIDE 6: SQL: COMPLEX JOINS & SUBQUERIES

- Join Taxonomy: INNER JOIN, LEFT/RIGHT OUTER JOIN, FULL OUTER JOIN, and CROSS JOIN operations
- Subquery Classifications: Nested subqueries, correlated subqueries, and scalar vs multi-row subqueries
- Set Operations: Combining query results using UNION, UNION ALL, INTERSECT, and EXCEPT
- Common Table Expressions (CTEs): Writing readable hierarchical queries using WITH clauses
- Performance implications: Correlated subquery execution cost compared to JOIN operations

VISUAL COMPANION COMPONENT SUGGESTION:

Visual Layout (Sleek Dark Mode): Venn diagrams representing the mathematical intersections of different SQL Joins.

LECTURE SPEAKER NOTES & PEDAGOGY:

Talking Points:

- Academic Formulation: Explain the mechanics of relational Joins: joining tables based on common key columns.
- Academic Formulation: Compare INNER JOIN (only matching keys) vs LEFT, RIGHT, and FULL OUTER JOINS.
- Academic Formulation: Discuss the CROSS JOIN (Cartesian product) and when its use is appropriate.
- Academic Formulation: Analyze Subqueries: compare nested subqueries vs correlated subqueries that reference parent table columns.
- Academic Formulation: Discuss SQL Set operators: UNION, UNION ALL, INTERSECT, and EXCEPT.
- Academic Formulation: Detail the syntax and performance benefits of Common Table Expressions (CTEs) for recursive queries.

Pedagogical Tip:

Use two overlapping circles to draw SQL Joins on the whiteboard, tracing output matching rows.

Classroom Examples:

- > "INNER JOIN: matching orders to customer profiles."
- > "LEFT JOIN: finding all customers, including those with zero orders."
- > "Correlated Subquery: finding employees who earn more than the average salary of their specific department."
- > "CTE: WITH regional_sales AS (SELECT region, SUM(amount) FROM sales GROUP BY region) SELECT * FROM regional_sales."

SLIDE 7: SQL: DATABASE INDEXES & QUERY EXECUTION

- Index Structures: B+ Tree indexes for range scans and sorting, Hash indexes for fast point lookups
- Clustered Index: Physical sorting order of rows on disk matches index order (one per table)
- Non-Clustered Index: Index contains pointers to actual data row addresses (multiple per table)
- Index Overhead: Speeding up read queries at the cost of slower write operations (INSERT/UPDATE)
- Index Selection Criteria: Analyzing columns cardinality and search conditions for index mapping

VISUAL COMPANION COMPONENT SUGGESTION:

Visual Layout (Sleek Dark Mode): Diagram of a B+ Tree node structure showing root node, internal routing nodes, and linked leaf nodes.

LECTURE SPEAKER NOTES & PEDAGOGY:

Talking Points:

- Academic Formulation: Define a Database Index: a secondary data structure designed to speed up search operations.
- Academic Formulation: Walk through B+ Tree index structures: self-balancing search trees storing key-pointer pairs.

- Academic Formulation: Explain why B+ Trees are preferred over B-Trees: leaf nodes are linked, allowing fast sequential scans.
- Academic Formulation: Differentiate between a Clustered Index (physical row ordering) and a Non-Clustered Index.
- Academic Formulation: Explain Hash Indexes: key-value mapping offering O(1) searches, but unsuitable for range queries.
- Academic Formulation: Discuss the write overhead of indexes: every INSERT, UPDATE, and DELETE statement must update the index structures.

Pedagogical Tip:

Show a book's index pages as a physical model of a database non-clustered index lookup.

Classroom Examples:

- > "Clustered Index: table sorted by ID on disk."
- > "Non-Clustered Index: index on Email column containing pointers to physical disk blocks."
- > "Hash Index: searching for exactly user_id = 12345 in a large user registry."
- > "Range Query scan: finding all orders created between Jan 1st and Jan 31st using a B+ Tree index."

SLIDE 8: SQL: QUERY EXECUTION PLANS & OPTIMIZATION

- Query Compiler: Parses query syntax, checks database catalogs, and runs algebraic optimizations
- Execution Plan: Blueprint detailing scan operations (Seq Scan, Index Scan) and Join types (Nested Loop, Hash Join)
- Explain Commands: Reviewing performance costs using EXPLAIN and EXPLAIN ANALYZE statements
- Optimization Strategies: Avoiding SELECT *, utilizing proper indexes, and writing efficient JOIN orders
- Statistics Collector: How the query planner uses column distribution histograms to estimate execution costs

VISUAL COMPANION COMPONENT SUGGESTION:

Visual Layout (Sleek Dark Mode): Execution plan tree nodes matching query operators to database access costs.

LECTURE SPEAKER NOTES & PEDAGOGY:**Talking Points:**

- Academic Formulation: Explain the components of the database Query Optimizer: parser, rewriter, cost estimator, and code generator.
- Academic Formulation: Discuss cost-based optimization (CBO): choosing plans based on disk I/O, CPU cycles, and table statistics.
- Academic Formulation: Explain how Seq Scan, Index Scan, and Index-Only Scans work under the hood.
- Academic Formulation: Compare Join execution plans: Nested Loop join, Hash Join, and Merge Join.
- Academic Formulation: Demonstrate how to read execution plan outputs using the EXPLAIN command.
- Academic Formulation: List optimization best practices: avoiding SELECT *, using query filters early, and updating optimizer statistics.

Pedagogical Tip:

Run a simulated EXPLAIN command on the board and ask students to identify where a table scan is slowing down execution.

Classroom Examples:

- > "Seq Scan vs Index Scan: scanning 10 million rows sequentially vs reading 2 index leaf blocks."
- > "Nested Loop: joining a tiny lookup table to a small customer table."
- > "Hash Join: joining two massive tables by creating an in-memory hash table for the right relation."
- > "Explain Output: showing a plan cost rating of 'rows=1000 width=45' in PostgreSQL."

SLIDE 9: COMPUTER NETWORKS: REFERENCE MODELS & TOPOLOGIES

- OSI Reference Model: Standardized 7-layer framework from Physical Layer to Application Layer
- TCP/IP Suite: Simplified 4-layer model (Link, Internet, Transport, Application) matching internet protocols
- Encapsulation: Wrapping payload data with headers/trailers at each layer (Frame, Packet, Segment, Message)
- Network topologies: Star, Mesh, Ring, and Bus structures determining structural reliability
- Network Metrics: Bandwidth, throughput, latency, jitter, and packet loss rates definitions

VISUAL COMPANION COMPONENT SUGGESTION:

Visual Layout (Sleek Dark Mode): Layer comparison columns mapping OSI 7 layers to TCP/IP 4 layers.

LECTURE SPEAKER NOTES & PEDAGOGY:

Talking Points:

- Academic Formulation: Introduce the concept of Computer Networks: interconnecting computing systems to share resources.
- Academic Formulation: Detail the OSI Reference Model: Physical, Data Link, Network, Transport, Session, Presentation, Application layers.
- Academic Formulation: Compare the OSI model with the practical 4-layer TCP/IP Suite (Link, Internet, Transport, Application).
- Academic Formulation: Explain data encapsulation: adding headers (IP, TCP, Ethernet) and trailers (CRC) at each stack boundary.
- Academic Formulation: Describe physical network topologies: star, bus, ring, and full-mesh connection layouts.
- Academic Formulation: Discuss network performance metrics: bandwidth, throughput, latency, jitter, and packet loss.

Pedagogical Tip:

Trace a single message's encapsulation path from an Application down to copper Ethernet wires on the board.

Classroom Examples:

- > "Application Layer payload: HTTP GET request message."
- > "Transport Layer segment: TCP segment with source and destination ports."
- > "Network Layer packet: IP packet with routing addresses."
- > "Link Layer frame: Ethernet frame with physical MAC addresses."

SLIDE 10: COMPUTER NETWORKS: PHYSICAL & DATA LINK LAYERS

- Transmission Media: Guided (fiber-optic, twisted pair) vs unguided (wireless radio, microwave) communication channels
- Framing & Flow Control: Chunking packets into physical frames and running Stop-and-Wait or Sliding Window flow rules
- Error Detection: Parity checks, Checksum calculations, and Cyclic Redundancy Checks (CRC)
- Medium Access Control: Managing shared medium conflicts via CSMA/CD (Ethernet) and CSMA/CA (Wi-Fi)
- Layer 2 Hardware: Forwarding frames using MAC address tables on multiport switches

VISUAL COMPANION COMPONENT SUGGESTION:

Visual Layout (Sleek Dark Mode): Layout diagram of an Ethernet Frame header, showing Preamble, MAC addresses, EtherType, Payload, and CRC fields.

LECTURE SPEAKER NOTES & PEDAGOGY:

Talking Points:

- Academic Formulation: Discuss the Physical Layer: modulating digital bits into electrical, light, or radio signals.
- Academic Formulation: Introduce guided media (fiber-optic cables, copper twisted pair) vs unguided wireless channels.
- Academic Formulation: Explain Data Link Layer framing: identifying frame starts and ends using bit stuffing.
- Academic Formulation: Discuss Error Detection: parity checks, internet checksums, and Cyclic Redundancy Checks (CRC) via polynomial division.
- Academic Formulation: Explain Medium Access Control (MAC): CSMA/CD for wired networks (detecting collision conflicts) and CSMA/CA for wireless networks.
- Academic Formulation: Differentiate between physical hardware devices: hubs (layer 1 repeating) vs switches (layer 2 MAC routing).

Pedagogical Tip:

Use the analogy of a group of people talking in a dark room to explain CSMA/CD collision detection.

Classroom Examples:

- > "Fiber-optic communication: using light pulses to transmit gigabits over continental distances."
- > "CRC check: a network card calculating the checksum of a frame and discarding a corrupted WiFi packet."
- > "CSMA/CD: a computer waiting for a silent line before transmitting, and sending a jam signal on collision."
- > "Switch lookup: a layer 2 switch forwarding a frame exclusively to port 4 based on its internal MAC address table."

SLIDE 11: COMPUTER NETWORKS: NETWORK LAYER & IP ROUTING

- IP Addressing: IPv4 32-bit addressing limitations and IPv6 128-bit hexadecimal addressing format
- Subnet Masking: Classless Inter-Domain Routing (CIDR) dividing network addresses and host addresses
- Routing Algorithms: Distance Vector Routing (RIP) vs Link State Routing (OSPF)
- Autonomous Systems: Border Gateway Protocol (BGP) managing inter-domain internet traffic routing
- NAT & DHCP: Dynamic Host Configuration Protocol and Network Address Translation mechanisms

VISUAL COMPANION COMPONENT SUGGESTION:

Visual Layout (Sleek Dark Mode): Interactive mapping chart showing path cost updates between neighboring routers.

LECTURE SPEAKER NOTES & PEDAGOGY:**Talking Points:**

- Academic Formulation: Explain the Network Layer's primary job: routing packets across heterogeneous network domains.
- Academic Formulation: Contrast IPv4 (32-bit addresses in dot-decimal notation) with IPv6 (128-bit hexadecimal notation).
- Academic Formulation: Explain Subnet Masking: applying bitwise AND operations to extract network IDs from IP addresses.
- Academic Formulation: Detail Classless Inter-Domain Routing (CIDR) notation and network allocation maps.
- Academic Formulation: Compare routing algorithms: Distance Vector (Bellman-Ford algorithm) vs Link State (Dijkstra's shortest path algorithm).
- Academic Formulation: Discuss Border Gateway Protocol (BGP): the path-vector routing protocol that connects Autonomous Systems across the global internet.

Pedagogical Tip:

Calculate a subnet ID from an IP (e.g. 192.168.1.55/24) on the board with the students.

Classroom Examples:

- > "IP mapping: CIDR address 10.0.0.0/16 reserving first 16 bits for network ID (65,534 hosts)."
- > "Dijkstra's algorithm: OSPF calculating the shortest path between routers based on line bandwidth metrics."
- > "BGP update: internet routers updating path lists when an optical cable in the Atlantic Ocean is cut."
- > "NAT routing: home routers mapping private IPs to a single public IP."

SLIDE 12: COMPUTER NETWORKS: TRANSPORT PROTOCOLS & CONGESTION CONTROL

- TCP vs UDP: Connection-oriented, reliable stream transmission vs connectionless, low-overhead datagram packet delivery
- TCP Connection: Establishing sessions via 3-way handshake (SYN, SYN-ACK, ACK) and closing via 4-way exchange
- Reliability mechanisms: Sequence numbers, cumulative acknowledgements, and Retransmission Timeouts (RTO)
- Congestion Control: Managing congestion window using Slow Start, Congestion Avoidance, Fast Retransmit, and Fast Recovery
- Socket Programming: End-to-end communication channels binding IP addresses and port numbers

VISUAL COMPANION COMPONENT SUGGESTION:

Visual Layout (Sleek Dark Mode): Sequence diagram charting a 3-way TCP handshake and subsequent packet transmissions.

LECTURE SPEAKER NOTES & PEDAGOGY:**Talking Points:**

- Academic Formulation: Introduce the Transport Layer: end-to-end logical communication between application processes.
- Academic Formulation: Compare TCP (connection-oriented, reliable, ordered stream) vs UDP (connectionless, lightweight datagram flow).
- Academic Formulation: Detail the TCP 3-way handshake: Client SYN -> Server SYN-ACK -> Client ACK.
- Academic Formulation: Explain TCP reliability: sequence numbers for ordering, checksums, and Retransmission Timeouts (RTO).
- Academic Formulation: Discuss TCP Sliding Window: flow control mechanism preventing the sender from overwhelming the receiver's buffer.
- Academic Formulation: Analyze TCP Congestion Control: managing network capacity using Slow Start, Congestion Avoidance, and Fast Recovery.

Pedagogical Tip:

Act out a 3-way handshake with a student to show how sequence numbers are synchronized.

Classroom Examples:

- > "UDP usage: online gaming or live video streaming where low latency matters more than lost frames."
- > "TCP usage: downloading a file or fetching a web page where data accuracy is critical."
- > "Handshake SYN flag: client sending SYN (seq=100) to server to initiate connection."
- > "Congestion collapse: network packets drop, triggers timeout, sender resends, worsening network congestion."

SLIDE 13: OOPS: CORE OOP CONCEPTS & CLASSES

- Object-Oriented Paradigm: Modelling systems as collections of autonomous objects combining data and behavior
- Encapsulation: Bundling data (attributes) and methods, and restricting access via private, protected, and public visibilities
- Abstraction: Hiding internal implementation complexities and exposing only essential interfaces to clients
- Classes vs Objects: A class acts as a compile-time blueprint, whereas an object is a runtime memory instance
- Constructor Overloading: Initializing objects with varying parameters lists configurations

VISUAL COMPANION COMPONENT SUGGESTION:

Visual Layout (Sleek Dark Mode): UML Class diagram detailing class attributes, methods, and access control specifiers.

LECTURE SPEAKER NOTES & PEDAGOGY:

Talking Points:

- Academic Formulation: Introduce the Object-Oriented Programming (OOP) paradigm, emphasizing structure and modularity.
- Academic Formulation: Define a Class: a user-defined blueprint containing attributes (state) and methods (behavior).
- Academic Formulation: Define an Object: an instance of a class allocated in memory during program runtime.
- Academic Formulation: Explain Encapsulation: wrapping data and methods, and restricting direct access via access modifiers.
- Academic Formulation: Discuss visibilities: public (global access), private (same class only), and protected (class and subclasses).
- Academic Formulation: Explain Abstraction: using interfaces or simple class declarations to hide complex implementation details.

Pedagogical Tip:

Draw a blueprint of a house (Class) vs actual houses built in a neighborhood (Objects) on the whiteboard.

Classroom Examples:

- > "Encapsulation: making employee salary private and exposing a public getSalary() method with access checks."
- > "Class declaration: public class Account { private double balance; public void deposit(double amt) {...} }."
- > "Object creation: Account myAcc = new Account();"
- > "Abstraction: using a car dashboard (steering wheel, pedals) without knowing how the internal combustion engine works."

SLIDE 14: OOPS: INHERITANCE MODELS & POLYMORPHISM

- Inheritance: Reusing code by deriving child classes from parent classes, establishing an 'is-a' relationship
- Inheritance types: Single, Multiple, Hierarchical, and Multi-level inheritance structures
- Compile-time Polymorphism: Function and operator overloading resolving method calls at compile time
- Runtime Polymorphism: Method overriding and dynamic dispatch resolving method executions during runtime
- Diamond Problem: Resolution methods (C++ virtual inheritance, Java interface contracts) for multiple inheritance

VISUAL COMPANION COMPONENT SUGGESTION:

Visual Layout (Sleek Dark Mode): Class inheritance hierarchy tree showing Base class and Derived subclasses with overridden methods.

LECTURE SPEAKER NOTES & PEDAGOGY:

Talking Points:

- Academic Formulation: Define Inheritance: mechanism where a child class acquires the properties and behaviors of a parent

class.

- Academic Formulation: Discuss inheritance hierarchies: parent-child structures establishing an 'is-a' relational link.
- Academic Formulation: Analyze Inheritance types: single, multilevel, hierarchical, and the diamond problem in multiple inheritance.
- Academic Formulation: Define Polymorphism: the ability of a single interface or message to represent different forms.
- Academic Formulation: Explain Compile-time Polymorphism: function and operator overloading resolved at compile time.
- Academic Formulation: Explain Runtime Polymorphism: method overriding, dynamic binding, and virtual function dispatch tables.

Pedagogical Tip:

Ask students how compilers resolve overridden methods when a parent reference points to a child object.

Classroom Examples:

- > "Inheritance: class Dog extends Animal (Dog is an Animal)."
- > "Method Overloading: calculateArea(int radius) vs calculateArea(int length, int width) in same class."
- > "Method Overriding: parent Animal.makeSound() print 'Generic Sound', child Cat.makeSound() print 'Meow'."
- > "Diamond Problem: class C inherits from A and B, which both inherit from Base, causing compiler ambiguity."

SLIDE 15: OOPS: INTERFACE ABSTRACTION & DESIGN PATTERNS

- Interfaces vs Abstract Classes: Interfaces define pure capabilities; abstract classes define shared behaviors and states
- Coupling & Cohesion: Aiming for loose coupling between components and high cohesion within component methods
- Creational Patterns: Factory Method, Singleton, and Builder patterns managing object instantiation logic
- Behavioral Patterns: Observer, Strategy, and Command patterns managing component communications
- Structural Patterns: Adapter and Decorator configurations adapting existing class classes interfaces

VISUAL COMPANION COMPONENT SUGGESTION:

Visual Layout (Sleek Dark Mode): Software architecture diagram representing the Observer pattern relationship.

LECTURE SPEAKER NOTES & PEDAGOGY:

Talking Points:

- Academic Formulation: Compare Abstract Classes vs Interfaces: abstract classes share state and methods; interfaces define pure behavior contracts.
- Academic Formulation: Discuss loose coupling: designing systems so components have minimal dependencies on each other's details.
- Academic Formulation: Explain high cohesion: grouping related functionalities tightly inside a single class or module.
- Academic Formulation: Introduce Creational Design Patterns: Factory Method for object creation, and Singleton for single-instance control.
- Academic Formulation: Introduce Structural Design Patterns: Adapter pattern mapping interface compatibilities.
- Academic Formulation: Introduce Behavioral Design Patterns: Observer pattern managing state change notifications across decoupled objects.

Pedagogical Tip:

Show how an electrical wall socket acts as a physical interface contract separating power production from appliances.

Classroom Examples:

- > "Interface contract: interface SecureConnection { void connect(); void disconnect(); }."
- > "Singleton pattern: database connection pool manager class ensuring only one manager instance exists."
- > "Factory method: creating different Document objects (PDF, Word) based on user selections."
- > "Observer pattern: newsletter subscription where users (observers) are notified of new posts (subject)."

SLIDE 16: OOPS: THE SOLID PRINCIPLES OF SOFTWARE DESIGN

- Single Responsibility & Open-Closed: A class should have one reason to change, and be open for extension but closed for modification
- Liskov Substitution: Subtypes must be substitutable for their base types without altering application correctness
- Interface Segregation: Clients should not be forced to depend on interface methods they do not use
- Dependency Inversion: High-level modules should depend on abstractions, not on concrete low-level implementations
- Design Review: Auditing complex OOP codebases against SOLID compliance metrics

VISUAL COMPANION COMPONENT SUGGESTION:

Visual Layout (Sleek Dark Mode): SOLID principles diagram mapping each letter to a clean class code example.

LECTURE SPEAKER NOTES & PEDAGOGY:

Talking Points:

- Academic Formulation: Introduce the SOLID principles of Object-Oriented design, formulated by Robert C. Martin.
- Academic Formulation: Explain the Single Responsibility Principle (SRP): a class should have only one reason to change.
- Academic Formulation: Detail the Open-Closed Principle (OCP): software entities should be open for extension but closed for modification.
- Academic Formulation: Explain the Liskov Substitution Principle (LSP): objects of a superclass should be replaceable with subclasses without breaking the app.
- Academic Formulation: Detail the Interface Segregation Principle (ISP): split bloated interfaces into smaller, client-specific interfaces.
- Academic Formulation: Explain the Dependency Inversion Principle (DIP): depend on abstractions (interfaces) rather than concrete implementations.

Pedagogical Tip:

Show how violating Liskov Substitution (e.g. Ostrich inheriting from FlyingBird) leads to runtime crashes.

Classroom Examples:

- > "SRP: separating user validation logic from user database persistence logic into separate classes."
- > "OCP: using a Strategy pattern to add new payment methods without modifying the central checkout class."
- > "LSP: a Rectangle class and a Square class; if Square violates Rectangle's height/width invariants, it violates LSP."
- > "DIP: a UserProfile class depending on a DatabaseInterface rather than a concrete PostgreSQLConnection class."

SLIDE 17: OPERATING SYSTEMS: KERNEL ARCHITECTURES & SCHEDULER

- Operating System: Software that manages computer hardware resources and provides common services for applications
- Kernel Architectures: Monolithic kernels (high speed, single memory space) vs Microkernels (modular, secure, IPC-reliant)
- Process States: Transitioning between New, Ready, Running, Waiting, and Terminated states
- CPU Scheduling: Scheduling algorithms including Round Robin, First-Come-First-Serve, Shortest Job First, and Priority Scheduling
- Context Switching: Saving and loading CPU register states in Process Control Blocks (PCBs) during scheduler execution

VISUAL COMPANION COMPONENT SUGGESTION:

Visual Layout (Sleek Dark Mode): Process State Transition diagram showing transitions and interrupt handlers.

LECTURE SPEAKER NOTES & PEDAGOGY:

Talking Points:

- Academic Formulation: Define the role of the Operating System: allocating hardware, managing memory, and providing a runtime environment.
- Academic Formulation: Compare Kernel architectures: Monolithic kernels vs microkernels.
- Academic Formulation: Introduce CPU Scheduling: the OS scheduler selecting ready processes from queue to execute on CPU cores.
- Academic Formulation: Explain the Process States: transitioning from New to Ready, Running, Waiting, and Terminated states.
- Academic Formulation: Discuss Preemptive vs Non-Preemptive scheduling algorithms.
- Academic Formulation: Analyze scheduling policies: Round Robin, First-Come-First-Serve, Shortest Job First, and Priority Scheduling.

Pedagogical Tip:

Draw the process state transition circle on the whiteboard, tracing timer interrupts and I/O waits.

Classroom Examples:

- > "Monolithic kernel: Linux containing device drivers, file system, and scheduler in kernel space."
- > "Microkernel: Mach or QNX running drivers and file systems as user-space daemons."
- > "Round Robin: scheduler allocating 10ms timeslices to active processes to provide multitasking."
- > "I/O Interrupt: process reading a file transitions from Running state to Waiting state until the disk read completes."

SLIDE 18: OPERATING SYSTEMS: THREADS & SYNC CONTROLS

- Processes vs Threads: A process is an independent execution unit with its own memory; a thread is a lightweight process sharing memory
- Race Conditions: Bugs occurring when concurrent threads modify shared variables without synchronization
- Critical Section: Thread code execution block that accesses shared memory and must be restricted to single-thread entry
- Synchronization Primitives: Mutual Exclusion (Mutex) locks, Semaphores (binary/counting), and Condition Variables
- Classic Sync Problems: Resolving the Bounded-Buffer, Readers-Writers, and Dining Philosophers challenges

VISUAL COMPANION COMPONENT SUGGESTION:

Visual Layout (Sleek Dark Mode): Thread synchronization diagram showing two threads using a Mutex to access a shared queue.

LECTURE SPEAKER NOTES & PEDAGOGY:**Talking Points:**

- Academic Formulation: Explain the difference between a Process (isolated memory space) and a Thread (lightweight unit sharing memory).
- Academic Formulation: Discuss multi-threaded programming benefits: responsiveness, resource sharing, and parallel execution.
- Academic Formulation: Explain Race Conditions: occurrences when multiple threads access and modify shared data concurrently.
- Academic Formulation: Define Critical Section: block of code accessing shared resources that must execute with mutual exclusion.
- Academic Formulation: Introduce Mutexes (Mutual Exclusion locks) to ensure only one thread enters the critical section.
- Academic Formulation: Explain Semaphores: counting semaphores managing resource pools, and binary semaphores acting as locks.

Pedagogical Tip:

Use the analogy of a bathroom key in a coffee shop to explain how a Mutex locks access to a shared resource.

Classroom Examples:

- > "Race Condition: two threads running `balance += 100` concurrently, resulting in lost updates due to interleaved execution."
- > "Mutex lock: thread calling `lock()` before incrementing a counter, and `unlock()` immediately after."
- > "Counting Semaphore: a semaphore initialized to 3 to manage access to a connection pool with 3 available database connections."
- > "Producer-Consumer problem: threads synchronization using semaphores to prevent buffer overflow/underflow."

SLIDE 19: OPERATING SYSTEMS: VIRTUAL MEMORY & PAGING

- Memory Hierarchy: Fast registers and cache, main physical memory (RAM), and slow secondary persistent storage
- Virtual Memory: Exposing a large, continuous address space to processes by translating addresses via Page Tables
- Paging: Dividing memory into fixed-size frames (physical) and pages (virtual) to eliminate external fragmentation
- Page Fault & Replacement: Disk retrieval loops triggered on missing frames, and replacement algorithms (LRU, FIFO)
- Thrashing: System performance degradation occurring when physical RAM is insufficient, causing continuous disk paging swaps

VISUAL COMPANION COMPONENT SUGGESTION:

Visual Layout (Sleek Dark Mode): Memory Translation diagram mapping Virtual Address to Physical Frame via a Page Table.

LECTURE SPEAKER NOTES & PEDAGOGY:**Talking Points:**

- Academic Formulation: Explain physical memory limitations and the necessity of virtual memory abstractions.
- Academic Formulation: Discuss Virtual Memory: separating logical user memory from physical memory space.
- Academic Formulation: Detail Paging: breaking physical memory into frames and virtual memory into pages.
- Academic Formulation: Explain Page Table: system structures translating virtual addresses to physical frame locations.
- Academic Formulation: Explain Page Faults: hardware interrupts triggered when a page referenced by the CPU is not loaded in RAM.
- Academic Formulation: Analyze Page Replacement algorithms: Least Recently Used (LRU), First-In-First-Out (FIFO), and Optimal replacement.

Pedagogical Tip:

Trace a virtual-to-physical address translation using a small Page Table on the board.

Classroom Examples:

- > "Virtual Address: CPU addressing page 4 offset 100, translated via page table to physical frame 12 offset 100."
- > "Page Fault: process requesting a page that resides in secondary swap space, triggering OS disk fetch."
- > "LRU algorithm: discarding the page that has not been accessed for the longest time to free up a frame."
- > "Thrashing: an OS spending more time swapping pages in and out of disk than executing processes due to low free RAM."

SLIDE 20: OPERATING SYSTEMS: DEADLOCK CONDITIONS & RESOLUTION

- Deadlock Definition: State where a set of processes are blocked because each holds a resource and waits for another resource
- Coffman Conditions: Mutual Exclusion, Hold and Wait, No Preemption, and Circular Wait (all must hold for deadlock)
- Deadlock Prevention: Disallowing one of the Coffman conditions during system configuration
- Deadlock Avoidance & Recovery: Utilizing Banker's Algorithm to verify safe execution states, or terminating deadlocked processes
- Resource Allocation Graph (RAG): Cycle detection analysis verifying deadlock presence in a running system

VISUAL COMPANION COMPONENT SUGGESTION:

Visual Layout (Sleek Dark Mode): Resource Allocation Graph (RAG) showing a circular wait dependency loop between processes.

LECTURE SPEAKER NOTES & PEDAGOGY:**Talking Points:**

- Academic Formulation: Define Deadlock: a permanent state of blocking where processes wait for resources held by each other.
- Academic Formulation: Outline the 4 Coffman conditions: Mutual Exclusion, Hold and Wait, No Preemption, and Circular Wait.
- Academic Formulation: Discuss Resource Allocation Graphs (RAG) and how loops in the graph represent deadlocks.
- Academic Formulation: Explain Deadlock Prevention: breaking one of the 4 Coffman conditions (e.g. requesting all resources at once).
- Academic Formulation: Explain Deadlock Avoidance: dynamically checking resource allocations using Banker's Algorithm.
- Academic Formulation: Discuss Deadlock Detection & Recovery: running cycle detection algorithms and aborting deadlocked processes.

Pedagogical Tip:

Ask students to analyze a crossroads traffic gridlock to explain how Hold-and-Wait and Circular-Wait occur.

Classroom Examples:

- > "Circular Wait: Process 1 holds Printer and waits for Scanner; Process 2 holds Scanner and waits for Printer."
- > "Hold and Wait: process requesting resource B while holding resource A, instead of releasing A first."
- > "Banker's Algorithm: OS verifying if allocating 2 tapes leaves enough tapes to guarantee at least one process finishes safely."
- > "Deadlock recovery: database system automatically terminating and rolling back the younger transaction to break a cycle."

SECTION 2: PEDAGOGICAL ASSESSMENT BANK

QUESTION 1 (MCQ) [BLOOM TIER: UNDERSTANDING]

Which normal form removes partial functional dependencies on candidate keys?

- (A) A) First Normal Form (1NF)
- (B) B) Second Normal Form (2NF)
- (C) C) Third Normal Form (3NF)
- (D) D) Boyce-Codd Normal Form (BCNF)

Correct Answer: B) Second Normal Form (2NF)

QUESTION 2 (MCQ) [BLOOM TIER: REMEMBERING]

In a DBMS, which ACID property ensures that once a transaction commits, changes are written to persistent storage and survive crashes?

- (A) A) Atomicity
- (B) B) Consistency
- (C) C) Isolation
- (D) D) Durability

Correct Answer: D) Durability

QUESTION 3 (MCQ) [BLOOM TIER: UNDERSTANDING]

What concurrency control protocol requires transactions to acquire locks in a growing phase and release them in a shrinking phase?

- (A) A) Timestamp Ordering
- (B) B) Two-Phase Locking (2PL)
- (C) C) Write-Ahead Logging (WAL)
- (D) D) Optimistic Concurrency Control

Correct Answer: B) Two-Phase Locking (2PL)

QUESTION 4 (MCQ) [BLOOM TIER: UNDERSTANDING]

Which index structure is optimal for range query scans and sorting operations in databases?

- (A) A) Hash Index
- (B) B) B+ Tree Index
- (C) C) Bitmap Index
- (D) D) Inverted Index

Correct Answer: B) B+ Tree Index

QUESTION 5 (MCQ) [BLOOM TIER: REMEMBERING]

In SQL, which statement is classified as a Data Definition Language (DDL) command?

- (A) A) INSERT
- (B) B) SELECT
- (C) C) CREATE
- (D) D) UPDATE

Correct Answer: C) CREATE

QUESTION 6 (MCQ) [BLOOM TIER: APPLYING]

Which SQL join returns all rows from the left table and the matched rows from the right table, filling with NULL values on unmatched right columns?

- (A) A) INNER JOIN
- (B) B) LEFT OUTER JOIN
- (C) C) RIGHT OUTER JOIN
- (D) D) FULL OUTER JOIN

Correct Answer: B) LEFT OUTER JOIN

QUESTION 7 (MCQ) [BLOOM TIER: UNDERSTANDING]

What is the primary difference between a Clustered Index and a Non-Clustered Index?

- (A) A) Clustered index can only store binary numbers
- (B) B) Clustered index physically sorts the data rows on disk
- (C) C) Non-clustered index can only exist on primary key columns
- (D) D) Non-clustered index slows down read queries

Correct Answer: B) Clustered index physically sorts the data rows on disk

QUESTION 8 (MCQ) [BLOOM TIER: REMEMBERING]

In SQL, what keyword is used to define a Common Table Expression (CTE)?

- (A) A) JOIN
- (B) B) WITH
- (C) C) GROUP BY
- (D) D) HAVING

Correct Answer: B) WITH

QUESTION 9 (MCQ) [BLOOM TIER: REMEMBERING]

Which network layer in the OSI reference model is responsible for packet routing, addressing, and path determination?

- (A) A) Physical Layer
- (B) B) Data Link Layer
- (C) C) Network Layer
- (D) D) Transport Layer

Correct Answer: C) Network Layer

QUESTION 10 (MCQ) [BLOOM TIER: REMEMBERING]

What error detection method is widely used at the Data Link layer by performing polynomial division on frame bits?

- (A) A) Simple Parity Check
- (B) B) 1s Checksum
- (C) C) Cyclic Redundancy Check (CRC)
- (D) D) Hamming Code

Correct Answer: C) Cyclic Redundancy Check (CRC)

QUESTION 11 (MCQ) [BLOOM TIER: APPLYING]

In computer networks, CIDR notation /24 corresponds to which subnet mask?

- (A) A) 255.0.0.0
- (B) B) 255.255.0.0
- (C) C) 255.255.255.0
- (D) D) 255.255.255.255

Correct Answer: C) 255.255.255.0

QUESTION 12 (MCQ) [BLOOM TIER: UNDERSTANDING]

Which Transport layer mechanism prevents a sender from overwhelming the receiver's incoming packet buffer?

- (A) A) 3-Way Handshake
- (B) B) Sliding Window Flow Control
- (C) C) Slow Start Congestion Avoidance
- (D) D) Checksum Validation

Correct Answer: B) Sliding Window Flow Control

QUESTION 13 (MCQ) [BLOOM TIER: REMEMBERING]

What OOP concept refers to bundling attributes and methods inside a single class and restricting direct user access?

- (A) A) Inheritance
- (B) B) Encapsulation
- (C) C) Polymorphism
- (D) D) Abstraction

Correct Answer: B) Encapsulation

QUESTION 14 (MCQ) [BLOOM TIER: UNDERSTANDING]

Which polymorphic mechanism enables method execution to be resolved at runtime using virtual function tables?

- (A) A) Function Overloading
- (B) B) Operator Overloading
- (C) C) Method Overriding
- (D) D) Class Inheritance

Correct Answer: C) Method Overriding

QUESTION 15 (MCQ) [BLOOM TIER: UNDERSTANDING]

In SOLID design principles, what does Liskov Substitution Principle (LSP) state?

- (A) A) A class should have only one reason to change
- (B) B) Subclasses must be substitutable for their base classes without breaking program correctness
- (C) C) High level modules should depend on concrete classes
- (D) D) Interfaces should be consolidated into a single monolithic API

Correct Answer: B) Subclasses must be substitutable for their base classes without breaking program correctness

QUESTION 16 (MCQ) [BLOOM TIER: REMEMBERING]

What design pattern guarantees that a class has only one instance and provides a global access point to it?

- (A) A) Factory Pattern
- (B) B) Strategy Pattern
- (C) C) Singleton Pattern
- (D) D) Observer Pattern

Correct Answer: C) Singleton Pattern

QUESTION 17 (MCQ) [BLOOM TIER: APPLYING]

Which CPU scheduling algorithm allocates a fixed processing timeslice (quantum) to each ready process in a cyclic order?

- (A) A) First-Come-First-Serve (FCFS)
- (B) B) Shortest Job First (SJF)
- (C) C) Round Robin (RR)
- (D) D) Priority Scheduling

Correct Answer: C) Round Robin (RR)

QUESTION 18 (MCQ) [BLOOM TIER: REMEMBERING]

What thread synchronization primitive acts as a lock, ensuring mutual exclusion by allowing only one thread to enter a critical section at a time?

- (A) A) Mutex
- (B) B) Counting Semaphore
- (C) C) Page Table
- (D) D) Thread Scheduler

Correct Answer: A) Mutex

QUESTION 19 (MCQ) [BLOOM TIER: REMEMBERING]

What hardware interrupt is triggered by the MMU when a process references a virtual memory page that is not currently loaded in physical RAM?

- (A) A) System Interrupt
- (B) B) Page Fault
- (C) C) Thread Context Switch
- (D) D) Segmentation Abort

Correct Answer: B) Page Fault

QUESTION 20 (MCQ) [BLOOM TIER: REMEMBERING]

Which of the following is NOT one of the four Coffman conditions required for a system deadlock to occur?

- (A) A) Mutual Exclusion
- (B) B) Hold and Wait
- (C) C) Preemption
- (D) D) Circular Wait

Correct Answer: C) Preemption